BBAT104: FUNDAMENTALS OF TQM

COURSE PROJECT GUIDELINES, ALLOTMENT & MARKING SCHEME

Academic Session: 2026–27 | Max. Assessment: 70 Raw Marks

---

1. PROJECT OVERVIEW & OBJECTIVES

The BBAT104 Course Project is an integrated software development and quality audit task. Every student is assigned a baseline system based on their Roll Number, along with a sequentially assigned Quality Goal (Q01 to Q20) within each project group. This structure yields up to 180 unique project combinations to guarantee individual accountability and deep hands-on application of Total Quality Management (TQM) principles.

2. ASSIGNMENT RULES & TOPIC MAPPING MATRIX

• **Baseline Software System Assignment Rule:** Assigned based on the last digit of the Roll Number, For a batch size of 200 students, the class is divided across 10 core baseline software topics (forming 10 working groups of approximately 20 students each). Every student within a group works on the assigned baseline system while implementing their unique sequential Quality Goal (Q01-Q20) on sequential basis.

Digit	Assigned Baseline Software System	Core TQM Goal Application
0	Attendance Management System	Automate tracking & minimize proxy entries
1	Library Management System	Optimize issue/return flow & catalog search speed
2	Hospital Management System	Reduce patient wait times & prevent record duplication
3	Inventory Management System	Eliminate stockouts & track turnover rate

4	Restaurant Billing System	Streamline order processing & error-free billing
5	Pharmacy Management System	Prevent expiry delivery & verify stock accuracy
6	Employee Management System	Automate payroll processing & performance logs
7	Hostel Management System	Optimize room allocation & complaint resolution
8	Parking Management System	Minimize entry queues & maximize space utilization
9	Online Quiz System	Ensure zero latency during submissions & auto-grading

3. ASSIGNED QUALITY GOALS MATRIX (Q01 – Q20)

• **Sequential Quality Goal Allotment Process:** While students in the same group share a common baseline system, each student executes an independent, sequentially assigned Quality Goal (Q01-Q20) to ensure 100% individual deliverable uniqueness.

- Students in Group 1 → roll no.'s ending with 0 (10,20,30,40,50……200) will be assigned quality goal (Q01-Q20) sequentially

- Student in Group 2→ roll no.'s ending with 1 (1, 11,21,31,41,51……191) will be assigned quality goal (Q01-Q20) sequentially

- Student in Group N → roll no.'s ending with N (N, -N, -N, -N, -N, -N……..--N) will be assigned quality goal (Q01-Q20) sequentially

Students allocated to a project group sequentially receive one of the 20 Quality Goals below. Students must select and implement any 5 suggested features associated with their assigned goal:

Code	Quality Goal Target	Suggested Features
Q01	Improve Reliability	Auto Backup, Audit Log, Input Validation, Error Recovery, User Roles
Q02	Improve Performance	Fast Search, Optimized Reports, Dashboard, CSV Import, Efficient DB Queries
Q03	Improve Usability	Dark Mode, Custom Themes, Dashboard Overview, Search Filters, Calendar
Q04	Improve Security	Role-Based Login, Password Reset, Input Validation, Audit Trail, Encryption
Q05	Reduce Response Time	Fast Search, Quick Navigation, Cached Dashboard, DB Indexing, Paginated Reports
Q06	Reduce Duplicate Entries	Unique ID Constraints, Pre-add Validation, Search Before Add, Confirmation Dialogs
Q07	Improve Data Accuracy	Input Masking, Dropdown Lists, Auto-Complete, Confirmation Modals, Audit Logs
Q08	Improve User Satisfaction	Interactive Dashboard, Visual Reports, Theme Toggle, Shortcuts, User Profile Settings
Q09	Reduce Bugs	Exception Handling, Strict Validation, Module Tests, Bug Tracker, Error Logs
Q10	Improve Documentation	Interactive README, User Manual, Help/FAQ Screen, About Page, Inline Code Comments
Q11	Improve Maintainability	Modular Structure, Config Files, System Settings, Activity Logs, Code Comments
Q12	Improve Accessibility	High Contrast, Font Resizing, Keyboard Navigation, Dark Mode, Minimalist Layout

Q13	Faster Search & Retrieval	Search Filters, Sorting Options, Dashboard Quick-Search, DB Query Optimization
Q14	Improve Navigation	Breadcrumb Trails, Top Navigation Bar, Shortcuts, Dashboard Tiles, Home Button
Q15	Better Reporting	PDF Export, Excel Export, Graphical Charts, Date Range Filters, Printable Layouts
Q16	Improve Testing	Unit Test Cases, Bug Log Matrix, Input Bounds Testing, System Logs
Q17	Reduce Memory Usage	Asset Compression, On-Demand Data Loading, DB Purge Tools, Query Caching
Q18	Improve Error Handling	Standardized Exception Catching, Friendly UI Alerts, System Error Logs, Recovery
Q19	Continuous Improvement (PDCA)	In-App Feedback Form, Bug Report Form, Version Log, PDCA Action Tracker
Q20	Overall Software Quality	Balanced Mix of Dashboard, Validation, Export Reports, Backups, and Role Controls

4. EVALUATION DELIVERABLES & MARKING SCHEME (70 RAW MARKS)

Milestone / Deliverable	**Raw Max Marks**	**Mapped COs**	**Evaluation Focus / Required Artefacts**
Review 1: Setup & SRS	10 Marks	CO1	GitHub Repo initialization, System Architecture Flowchart, SRS Document, Scope

			definition.
Review 2: Base System & CRUD	15 Marks	CO1, CO2	Base CRUD Modules functioning + execution of assigned 5 Quality Goal features.
Review 3: FMEA & Risk Audit	15 Marks	CO2	FMEA Matrix with RPN scores, SIPOC Process Map, CTQ Tree, Defect Logging.
Review 4: SQC & Continuous Improvement	15 Marks	CO2, CO3	Pareto Chart (80/20 analysis), Fishbone Diagram (Root Cause), Checksheets, PDCA Cycle log.
Final Demonstration & Viva	10 Marks	CO3	Live software demonstration, bug-handling defense, oral viva on TQM tools selection.
Documentation & GitHub Health	5 Marks	CO3	Comprehensive README.md, User Manual, >= 30 meaningful commit history, GitHub Issues.
Total Project Assessment	70 Marks	CO1, CO2, CO3	Total Raw Evaluation Score across all 4 reviews, demo, and documentation.

5. PERFORMANCE RUBRICS (4-TIER EVALUATION SCALE)

Criterion / CO	**Excellent (80%-100%)**	**Good (60%-79%)**	**Satisfactory (40%-59%)**	**Needs Work (<40%)**
System Architecture & Base CRUD (CO1)	All CRUD modules fully functional; robust architecture with zero breaking errors.	All CRUD modules present with minor UI/UX issues; good structural flow.	Core CRUD modules working; basic UI without full exception handling.	Incomplete CRUD modules; system prone to crashes.
Risk Analysis &	Accurate RPN	FMEA matrix	FMEA created	FMEA matrix

FMEA (CO2)	calculations; logical Severity/Occurrence ratings; actionable mitigation plan.	complete; minor gaps in RPN risk prioritization or mitigation strategies.	but missing core failure modes or inaccurate RPN ratings.	incomplete or copied without relevance to project.
SQC Tools & PDCA (CO2/CO3)	Flawless Pareto & Fishbone diagrams; clear data checksheets; verified PDCA log.	Valid SQC tools used; minor formatting or data alignment gaps in Fishbone/Pareto.	Basic SQC tools generated with superficial data; PDCA cycle incomplete.	Missing SQC diagrams or incorrect execution of Pareto/Fishbone tools.
Defense, Viva & GitHub (CO3)	Confident viva defense; >=30 commits; interactive README with screenshots and manual.	Solid viva defense; 20-29 commits; complete README with minor omissions.	Satisfactory viva; 10-19 commits; basic README without setup instructions.	Poor viva defense; <10 commits; missing or default README file.

6. REQUIRED TOOLS, SOFTWARE DEPENDENCIES, TQM PRINCIPLES & SQC TOOLS

6.1 Development Tools & Software Dependencies

• **Visual Studio / Visual Studio Code:** Primary Integrated Development Environment (IDE) used for writing, debugging, and managing source code.

• **GitHub Account:** Cloud repository platform for hosting project source code, version history, and submitting progress links.

• **GitHub Desktop:** GUI client used to sync local code commits, track version changes, and manage repository pushes/pulls.

• **Python & Core Dependencies:** Python 3.x core runtime alongside standard/third-party libraries: Tkinter/CustomTkinter (GUI interfaces), SQLite3/Pandas (Data persistence & storage), and Matplotlib/Seaborn (Automated generation of SQC charts).

6.2 Core TQM Principles Applied

• **Customer Focus:** Designing system features specifically to solve user operational pain points.
• **Continuous Improvement (Kaizen):** Iteratively refining software features through systematic PDCA feedback cycles.
• **Process-Centric Approach:** Mapping software inputs and outputs explicitly through structured SIPOC diagrams.
• **Fact-Based Decision Making:** Utilizing empirical defect tracking metrics to target bug fixes.
• **Error Prevention (Poka-Yoke):** Embedding strict input validation and field masks to eliminate errors at the entry level.

6.3 Statistical Quality Control (SQC) Tools Requirements

• **Pareto Analysis (80/20 Rule):** Charting defect frequencies to isolate the vital 20% causes driving 80% of system errors.
• **Ishikawa (Fishbone) Diagram:** Categorizing root causes across People, Process, Software Code, and Infrastructure.
• **Checksheets & Defect Logs:** Standardized logging matrices for tracking defect occurrence, severity, and resolution status.
• **FMEA Matrix:** Calculating Risk Priority Numbers (RPN = Severity × Occurrence × Detection) to prioritize fixes.

7. STEP-BY-STEP PROJECT EXECUTION PROCESS

• **Step 1: Environment Setup & Repository Initialization:** Install Python 3.x, Visual Studio Code, and GitHub Desktop. Create a public GitHub repository named 'BBAT104_TQM_[RegNo]' and commit an initial README.md containing project details.

• **Step 2: Requirement Analysis & SRS Documentation:** Identify assigned baseline system from Roll No digit. Draw a System Architecture Flowchart, establish Critical to Quality (CTQ) parameters, and prepare the SRS document.

• **Step 3: Base System & CRUD Development:** Develop the base application in VS Code using Python. Implement core CRUD (Create, Read, Update, Delete) operations connected to SQLite or file storage.

• **Step 4: Integration of Assigned Quality Goal Features:** Select any 5 features from your assigned Quality Goal (Q01–Q20). Program and integrate these specialized modules into your base application.

• **Step 5: Process Mapping & Risk Audit (FMEA):** Construct a SIPOC diagram mapping input-output flows. Build an FMEA matrix identifying failure modes, calculating RPN scores, and defining mitigation steps.

• **Step 6: Statistical Quality Control (SQC) & Continuous Improvement:** Log software bugs on a Checksheet. Generate Pareto Charts and Fishbone Diagrams in Python/Matplotlib. Execute the Plan-Do-Check-Act (PDCA) cycle.

• **Step 7: GitHub Version Control Sync & Documentation:** Commit code changes regularly via GitHub Desktop (minimum 30 commits). Update README.md with system architecture diagrams, user manuals, and setup instructions.

• **Step 8: Final Review Submission & Viva Defense:** Submit the completed repository link. Deliver a live software demonstration showing feature execution and defend TQM tool selections during oral viva.